

IS210.Q23 - HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU

SEMINAR GIẢI QUYẾT CÁC VẤN ĐỀ TRUY XUẤT ĐỒNG THỜI

MSSV	Họ tên
24520769	Trần Huỳnh Gia Khang
24520902	Lê Quốc Kiệt
24521296	Lâm Nguyên Phát
24521369	Bành Xuân Phúc

I. Giới thiệu

Trong bối cảnh các hệ thống thông tin hiện đại ngày càng đòi hỏi khả năng xử lý dữ liệu quy mô lớn, việc quản lý truy xuất đồng thời (Concurrency Control) đã trở thành một trong những thách thức cốt lõi của Hệ quản trị cơ sở dữ liệu. Đối với Oracle Database, mục tiêu tối thượng của kiểm soát đồng thời là đảm bảo tính toàn vẹn và nhất quán của dữ liệu trong môi trường có hàng nghìn giao dịch diễn ra đồng thời mà không làm suy giảm hiệu năng hệ thống.

Vấn đề nảy sinh khi nhiều tiến trình cùng tác động lên một tài nguyên duy nhất, dẫn đến các hiện tượng xung đột dữ liệu như: Đọc dữ liệu rác (Dirty Read), Đọc không lặp lại (Non-repeatable Read), Đọc ảo (Phantom Read) hay Bế tắc (Deadlock). Để giải quyết triệt để các rủi ro này, Oracle áp dụng cơ chế Kiểm soát truy xuất đồng thời đa phiên bản (Multi-Version Concurrency Control - MVCC). Thay vì sử dụng các cấu trúc khóa cứng nhắc gây nghẽn mạch hệ thống, Oracle tận dụng dữ liệu hoàn tác và số thứ tự thay đổi hệ thống để cung cấp các bản sao dữ liệu nhất quán tại một thời điểm cụ thể cho mỗi truy vấn.

Việc hiểu rõ cách thức Oracle thực thi các cấp độ cô lập theo tiêu chuẩn ANSI/ISO bao gồm Read Committed và Serializable là yếu tố tiên quyết để thiết kế các ứng dụng an toàn và hiệu quả.

II. Lý thuyết

1. Mức cô lập Read Committed

Mức cô lập Read Committed là mức cô lập giao dịch mặc định trong Oracle Database, phù hợp cho các môi trường cơ sở dữ liệu có ít khả năng xảy ra xung đột giữa các giao dịch. Đặc điểm của mức cô lập này là mỗi truy vấn do một giao dịch thực thi chỉ nhìn thấy dữ liệu đã được commit trước khi truy vấn đó bắt đầu, chứ không phải trước khi toàn bộ giao dịch bắt đầu.

Trong mức Read Committed, Oracle cung cấp tính nhất quán đọc ở cấp độ câu lệnh. Thời điểm nhất quán của dữ liệu được xác định là lúc câu lệnh được mở. Ví dụ: nếu một câu lệnh SELECT mở tại System Change Number (SCN) 1000, thì câu lệnh đó sẽ nhất quán với dữ liệu tính đến đúng SCN 1000. Tuy nhiên, nếu danh sách SELECT có chứa một hàm PL/SQL, tính nhất quán đọc sẽ được áp dụng ở cấp độ câu lệnh cho đoạn mã SQL chạy bên trong hàm đó (với mỗi lần thực thi hàm tạo ra một snapshot mới), chứ không phải áp dụng ở cấp độ của câu lệnh SQL cha.

Một truy vấn trong giao dịch Read Committed sẽ tránh được việc đọc phải những dữ liệu đang được một giao dịch khác thay đổi nhưng chưa commit. **Oracle Database không bao giờ cho phép xảy ra hiện tượng Dirty Read.**

2. Mức cô lập Serializable

Mức cô lập Serializable là mức cô lập được thiết kế để tạo ra một môi trường hoạt động mà ở đó các giao dịch dường như diễn ra tuần tự (serial), như thể không có người dùng nào khác đang sửa đổi dữ liệu cùng một lúc.

Khác với mức cô lập Read Committed (chỉ nhất quán tại thời điểm bắt đầu từng câu lệnh), trong mức Serializable, một giao dịch chỉ nhìn thấy những thay đổi đã được commit tại thời điểm toàn bộ giao dịch bắt đầu, cộng với những thay đổi do chính giao dịch đó thực hiện. Tính nhất quán đọc này kéo dài trong suốt quá trình giao dịch diễn ra. Bất kỳ hàng dữ liệu nào được giao dịch đọc đều được đảm bảo sẽ trả về cùng một kết quả.

nếu được truy vấn lại trong cùng giao dịch đó. Dù giao dịch có kéo dài bao lâu, các thay đổi do các giao dịch khác mới thực hiện cũng sẽ không hiển thị đối với các truy vấn của nó

3. Hiện tượng Lost Update

Lost Update xảy ra khi một giao dịch ghi ghi đè lên các thay đổi của một giao dịch ghi đồng thời khác trên cùng một phần dữ liệu. Vấn đề này thường phát sinh khi một giao dịch thao tác dựa trên những dữ liệu chưa được commit của một giao dịch khác.

Cơ chế xảy ra Lost Update (Thường gặp ở mức cô lập Read Committed) Lost Update là một tình huống kinh điển có thể xảy ra ở mức độ cô lập giao dịch mặc định là Read Committed. Hiện tượng này diễn ra qua các bước sau khi có sự xung đột ghi (Conflicting Writes):

- Bước 1: Giao dịch 1 (T1) tiến hành cập nhật một hàng dữ liệu nhưng chưa thực hiện lệnh commit. Khi đó, T1 sẽ nắm giữ một khóa hàng độc quyền trên hàng đó.
- Bước 2: Giao dịch 2 (T2) cũng cố gắng cập nhật chính hàng dữ liệu này. Do T1 đang giữ khóa, T2 sẽ bị chặn và buộc phải chờ cho đến khi T1 kết thúc.
- Bước 3: T1 thực hiện lệnh commit, làm thay đổi trên hàng dữ liệu trở thành vĩnh viễn và giải phóng khóa.
- Bước 4: Khóa được giải phóng, T2 thoát khỏi trạng thái chờ và ngay lập tức áp dụng bản cập nhật của chính nó lên hàng dữ liệu vừa được T1 sửa đổi.
- Bước 5: T2 commit. Lúc này, toàn bộ thay đổi mà T1 đã thực hiện trước đó bị T2 ghi đè hoàn toàn. Việc cập nhật của T1 coi như "bị mất".

Tuy nhiên, Mức cô lập Serializable không gặp phải hiện tượng Lost Update vì nó áp dụng một cơ chế kiểm soát cực kỳ nghiêm ngặt đối với thao tác sửa đổi dữ liệu, không cho phép các giao dịch âm thầm ghi đè lên nhau.

- Oracle Database chỉ cho phép một giao dịch Serializable sửa đổi (update/delete) một hàng dữ liệu nếu mọi thay đổi trên hàng đó (do các giao dịch khác thực hiện) đã được commit trước khi giao dịch Serializable này bắt đầu.
- Nếu giao dịch Serializable cố gắng thay đổi một hàng dữ liệu mà hàng này vừa được một giao dịch khác sửa đổi và commit *sau* thời điểm nó bắt đầu, hệ thống sẽ lập tức chặn thao tác này và báo lỗi ORA-08177: Cannot serialize access for this transaction.

Nhờ cơ chế báo lỗi thay vì ghi đè này, thay vì chờ đợi để áp dụng bản cập nhật đè lên dữ liệu của người khác (nguyên nhân gây ra Lost Update ở mức Read Committed), giao dịch Serializable sẽ bị lỗi và ứng dụng buộc phải xử lý (như rollback và thực hiện lại giao dịch). Điều này giúp dữ liệu luôn an toàn và tránh được hiện tượng Lost Update.

4. Hiện tượng Non – repeatable Read

Non-repeatable Read xảy ra khi một giao dịch thực hiện đọc cùng một hàng dữ liệu nhiều lần nhưng nhận về các kết quả khác nhau giữa các lần đọc. Vấn đề này phát sinh khi một giao dịch khác sửa đổi và commit dữ liệu đó ngay trong khoảng thời gian giữa hai lần truy vấn của giao dịch đầu tiên.

Cơ chế xảy ra Non-repeatable Read (Thường gặp ở mức cô lập Read Committed) Trong Oracle Database, mức cô lập mặc định là Read Committed đảm bảo tính nhất quán ở cấp độ câu lệnh nhưng không đảm bảo tính nhất quán ở cấp độ giao dịch. Hiện tượng này diễn ra qua các bước sau:

- Bước 1: Giao dịch 1 (T1) thực hiện truy vấn SELECT để đọc một hàng dữ liệu. Oracle sử dụng SCN tại thời điểm câu lệnh bắt đầu để truy xuất dữ liệu từ các Data Blocks hoặc Undo Segments.

- Bước 2: Giao dịch 2 (T2) thực hiện lệnh UPDATE trên chính hàng dữ liệu mà T1 vừa đọc, sau đó thực hiện lệnh COMMIT. Lúc này, dữ liệu trên đĩa và SCN của hệ thống đã được cập nhật mới.
- Bước 3: Giao dịch 1 (T1) thực hiện lại chính câu lệnh SELECT đó một lần nữa trong cùng một phiên làm việc.
- Bước 4: Do ở mức Read Committed, mỗi câu lệnh SELECT mới sẽ lấy một SCN mới. Oracle ghi nhận phiên bản dữ liệu đã được T2 commit và trả về kết quả mới cho T1.
- Bước 5: T1 kết thúc với hai kết quả khác nhau cho cùng một truy vấn, gây ra sự mất nhất quán trong logic xử lý của ứng dụng.

Tuy nhiên, Mức cô lập Serializable hoàn toàn loại bỏ hiện tượng Non-repeatable Read nhờ vào cơ chế Snapshot Isolation được hỗ trợ bởi kiến trúc đa phiên bản (MVCC) của Oracle:

- Khi một giao dịch được thiết lập ở mức Serializable, Oracle đóng băng góc nhìn dữ liệu của giao dịch đó tại thời điểm nó bắt đầu. Mọi câu lệnh SELECT bên trong giao dịch này sẽ luôn thấy dữ liệu y hệt nhau, bất chấp các giao dịch khác có commit thay đổi hay không.
- Nếu một hàng dữ liệu bị giao dịch khác thay đổi, Oracle sẽ tự động tìm vào vùng dữ liệu hoàn tác để tái cấu trúc lại phiên bản dữ liệu cũ phù hợp với SCN lúc T1 bắt đầu.

Nhờ cơ chế này, thay vì nhìn thấy dữ liệu mới nhất đã commit (như Read Committed), giao dịch Serializable luôn làm việc với một ảnh chụp dữ liệu cố định. Điều này giúp đảm bảo tính toàn vẹn dữ liệu tuyệt đối cho các báo cáo tài chính hoặc các tiến trình tính toán phức tạp cần sự chính xác xuyên suốt nhiều bước truy vấn.

5. Hiện tượng Phantom Read

Phantom Read xảy ra khi một giao dịch thực hiện truy vấn trên một tập hợp các hàng thỏa mãn một điều kiện lọc nhất định, nhưng trong các lần đọc sau đó, tập kết quả trả về bị thay đổi (xuất hiện thêm hàng mới hoặc mất đi hàng cũ). Khác với Non-repeatable Read tập trung vào sự thay đổi giá trị của một hàng cụ thể, Phantom Read liên quan đến sự biến động về số lượng bản ghi trong một vùng dữ liệu.

Cơ chế xảy ra Phantom Read (Thường gặp ở mức cô lập Read Committed) Trong Oracle Database, mức cô lập Read Committed chỉ bảo vệ tính nhất quán trên những hàng dữ liệu hiện hữu tại thời điểm câu lệnh bắt đầu, chứ không khóa toàn bộ vùng điều kiện của truy vấn. Hiện tượng này diễn ra như sau:

- Bước 1: Giao dịch 1 (T1) thực hiện một truy vấn tìm kiếm theo phạm vi, ví dụ: `SELECT * FROM NhanVien WHERE Luong > 5000`. Oracle trả về danh sách các nhân viên thỏa mãn điều kiện tại SCN hiện tại.
- Bước 2: Giao dịch 2 (T2) tiến hành chèn thêm một hàng dữ liệu mới (INSERT) thỏa mãn điều kiện trên (ví dụ: nhân viên mới có lương 6000) và thực hiện lệnh COMMIT.
- Bước 3: Giao dịch 1 (T1) thực hiện lại chính câu lệnh truy vấn đó một lần nữa.
- Bước 4: Do T2 đã commit, Oracle cấp một SCN mới cho câu lệnh thứ hai của T1. Câu lệnh này quét toàn bộ bảng và nhận diện được hàng dữ liệu mới mà T2 vừa chèn vào.
- Bước 5: T1 nhận thấy tập kết quả có sự xuất hiện của một "bóng ma" không tồn tại ở lần đọc đầu tiên, gây sai lệch cho các phép tính toán tổng hợp (SUM, COUNT) hoặc logic nghiệp vụ.

Tuy nhiên, Mức cô lập Serializable ngăn chặn hoàn toàn hiện tượng Phantom Read nhờ vào sự kết hợp giữa kiến trúc đa phiên bản (MVCC) và cơ chế Snapshot Isolation:

- Ngay khi giao dịch Serializable bắt đầu, Oracle xác định một ảnh chụp của toàn bộ cơ sở dữ liệu dựa trên SCN tại thời điểm đó. Bất kỳ hàng dữ liệu nào được chèn mới sau SCN này sẽ được coi là không tồn tại đối với giao dịch hiện tại.
- Khi T1 thực hiện lại truy vấn, Oracle sẽ kiểm tra SCN của các hàng dữ liệu. Nếu một hàng có SCN lớn hơn SCN bắt đầu của T1 (nghĩa là hàng đó được commit sau khi T1 bắt đầu), Oracle sẽ tự động loại bỏ hàng đó khỏi tập kết quả trả về.

Nhờ việc duy trì tính nhất quán ở cấp độ giao dịch, Oracle đảm bảo rằng tập hợp các hàng thỏa mãn điều kiện lọc sẽ luôn bất biến trong suốt vòng đời của một giao dịch Serializable. Điều này giúp loại bỏ sự xuất hiện của các bản ghi ảo, đảm bảo tính chính xác tuyệt đối cho các tiến trình xử lý dữ liệu hàng loạt.

6. Hiện tượng Deadlock

Deadlock là tình huống xảy ra khi hai hoặc nhiều giao dịch đang chờ đợi để truy cập vào các dữ liệu mà đối phương đang khóa, khiến cho các giao dịch này không thể tiếp tục thực thi.

Deadlock xảy ra khi có sự cạnh tranh tài nguyên theo một vòng lặp khép kín.

- Bước 1: Giao dịch 1 (T1) cập nhật một hàng dữ liệu A và hệ thống cấp cho T1 một khóa độc quyền trên hàng này. Đồng thời, Giao dịch 2 (T2) cập nhật một hàng dữ liệu B và cũng được cấp khóa độc quyền trên hàng B. Tới đây, cả hai giao dịch vẫn hoạt động bình thường.
- Bước 2: T1 tiếp tục muốn cập nhật hàng B. Vì T2 đang giữ khóa hàng B, T1 buộc phải đưa vào trạng thái chờ.
- Bước 3: Cùng lúc đó, T2 lại cố gắng cập nhật hàng A. Vì T1 đang giữ khóa hàng A, T2 cũng bị đưa vào trạng thái chờ.
- Kết quả: T1 chờ T2, và T2 lại chờ T1. Không giao dịch nào có thể lấy được tài nguyên mình cần để hoàn tất công việc và giải phóng khóa. Bất kể hai giao dịch này chờ bao lâu, các khóa xung đột vẫn sẽ bị giữ chặt, tạo thành một "bế tắc".

Khác với một số lỗi chỉ phát hiện được khi biên dịch câu lệnh, deadlock chỉ có thể bị phát hiện trong quá trình thực thi. Oracle Database xử lý tình huống này hoàn toàn tự động theo các bước:

- Cơ sở dữ liệu tự động phát hiện tình trạng bế tắc và phá vỡ nó bằng cách tự động rollback ở cấp độ câu lệnh đối với một trong số các câu lệnh đang gây ra xung đột. Việc hủy câu lệnh này giúp giải phóng một tập hợp khóa, cho phép giao dịch kia tiếp tục hoạt động.
- Giao dịch bị chọn để hoàn tác câu lệnh sẽ nhận được thông báo lỗi: ORA-00060: deadlock detected while waiting for resource. Lưu ý rằng hệ thống chỉ báo lỗi này cho một phiên (session) duy nhất trong số các phiên đang bị bế tắc, và việc chọn phiên nào để báo lỗi là ngẫu nhiên.
- Lỗi này chỉ gây ra hoàn tác ở cấp độ câu lệnh đối với chính câu lệnh sinh ra deadlock, các câu lệnh thay đổi dữ liệu đã thực hiện thành công trước đó của giao dịch bị báo lỗi vẫn được giữ nguyên và không bị rollback.

Khi một giao dịch nhận được lỗi ORA-00060, Oracle khuyến nghị ứng dụng hoặc người dùng nên chủ động gọi lệnh ROLLBACK một cách rõ ràng để hoàn tác toàn bộ giao dịch đó. Tuy nhiên, ứng dụng cũng có thể được lập trình để chờ một khoảng thời gian rồi thử thực thi lại câu lệnh vừa bị lỗi.

III. Các tình huống lỗi cụ thể

1. Lost update

Tình huống xảy ra lỗi: Hệ thống bán vé sự kiện Ve'ryGood có hỗ trợ tính năng hủy vé và hoàn tiền theo chính sách, số tiền được hoàn trả sẽ phụ thuộc vào thời điểm hủy vé, nếu hủy vé sớm (trước 2 tuần diễn ra sự kiện thì số tiền được hoàn trả là 100% tiền vé).

Một người tận dụng tính năng hoàn tiền và lỗi truy xuất lost update để trục lợi, anh ta chuẩn bị từ trước một vé mua sớm ở sự kiện khác với giá 100.000đ, biết ở thời điểm hiện tại nếu hủy vé này và hoàn tiền, anh ta sẽ nhận lại 100% tiền vé. Tiếp theo anh ta mua vé

concert Anh Trai Say Hi hạng vé PREMIER LOUNGE với giá 10.000.000đ, cùng lúc đó mở một tab khác chọn hoàn tiền tấm vé trị giá 100.000đ. Kết quả do lỗi lost update, anh ta mua được vé PREMIER LOUNGE của Anh Trai Say Hi mà không mất tiền, giả sử mã tài khoản của người này là 1, phiên 1 là tab anh ta dùng để mua vé Concert Anh Trai Say Hi, phiên 2 là tab anh ta dùng để hoàn tiền tấm vé 100.000đ, chi tiết xử lý bên dưới hệ thống như sau:

Phiên 1	Phiên 2	Giải thích
<pre>SQL> SELECT SoDu INTO v_SoDuHienTai FROM VI_CA_NHAN WHERE MaTaiKhoan = 1; DBMS_OUTPUT.PUT_LI NE('Số dư hiện tại là: ' v_SoDuHienTai); _____ Số dư hiện tại là: 10000000</pre>	Không có hoạt động	Phiên 1 truy vấn số dư trong ví của tài khoản 1
<pre>SQL> UPDATE VI_CA_NHAN SET SoDu = v_SoDuHienTai - 10000000 WHERE MaTaiKhoan = 1;</pre>	Không có hoạt động	Phiên 1 bắt đầu một transaction bằng cách cập nhật số dư trong ví của tài khoản 1. Số dư mới bằng số dư hiện tại trừ đi 10.000.000đ. Ngay lúc này, dòng dữ liệu có mã tài khoản bằng 1 trong Ví Cá

		Nhân bị khoá bởi transaction 1. Mức cô lập mặc định cho transaction này là READ COMMITTED
Không có hoạt động	SQL> SET TRANSACTION ISOLATION LEVEL READ COMMITTED;	Phiên 2 bắt đầu một transaction 2 và thiết lập tường minh mức cô lập READ COMMITTED
Không có hoạt động	SQL> SELECT SoDu INTO v_SoDuHienTai FROM VI_CA_NHAN WHERE MaTaiKhoan = 1; DBMS_OUTPUT.PUT_LINE('Số dư hiện tại là: ' v_SoDuHienTai); _____ Số dư hiện tại là: 10000000	Transaction 2 truy vấn số dư ví của tài khoản 1. Oracle Database sử dụng tính nhất quán đọc để hiển thị, do thao tác cập nhật của transaction 1 chưa được commit nên số dư hiển thị ở phiên 2 là số dư trước khi transaction 1 bắt đầu.
Không có hoạt động	SQL> UPDATE VI_CA_NHAN SET SoDu = v_SoDuHienTai + 100000 WHERE MaTaiKhoan = 1; — Câu lệnh không trả về	Transaction 2 cố gắng cập nhật số dư của tài khoản 1 tăng thêm 100.000, tuy nhiên dòng dữ liệu này đã bị khoá bởi transaction 1, điều này tạo ra một xung đột ghi. Transaction 2 phải chờ đến

		khi transaction 1 kết thúc mới được thực thi tiếp
SQL> COMMIT;	Không có hoạt động	Transaction 1 commit và kết thúc
Không có hoạt động	1 row affected. SQL>	Khoá trên dòng dữ liệu số dư tài khoản 1 giờ đã được giải phóng, nên transaction 2 tiếp tục cập nhật số dư ví cho tài khoản 1
Không có hoạt động	SQL> SELECT SoDu INTO v_SoDuHienTai FROM VI_CA_NHAN WHERE MaTaiKhoan = 1; DBMS_OUTPUT.PUT_LINE('Số dư hiện tại là: ' v_SoDuHienTai); _____ Số dư hiện tại là: 10100000	Transaction 2 truy vấn số dư trong ví của tài khoản 1. Transaction 2 thấy được sự cập nhật của nó trên số dư ví của tài khoản 1.
Không có hoạt động	SQL> COMMIT;	Transaction 2 COMMIT và kết thúc
SQL> SELECT SoDu INTO v_SoDuHienTai FROM VI_CA_NHAN WHERE MaTaiKhoan = 1;	Không có hoạt động	Phiên 1 truy vấn số dư trong tài khoản 1. Số dư lúc này là 10.100.000, đây là con số được cập nhật bởi transaction 2. Sự cập nhật số dư tài khoản 1 trừ đi

DBMS_OUTPUT.PUT_LINE('Số dư hiện tại là: ' v_SoDuHienTai); Số dư hiện tại là: 10100000		10.000.000 bởi transaction 1 giờ đã bị 'mất'
--	--	---

Để khắc phục triệt để lỗi hỏng Lost Update trong tình huống thanh toán và hoàn tiền đồng thời này, giải pháp được đưa ra là thiết lập mức cô lập của các giao dịch lên **SERIALIZABLE**. Thông qua việc ép buộc các giao dịch xung đột phải báo lỗi và chạy lại thay vì ghi đè lẫn nhau, Oracle Database đảm bảo rằng mọi thay đổi về số dư ví đều được áp dụng tuần tự. Kết quả là hệ thống bảo toàn được cả hai thao tác tài chính, số dư cuối cùng phản ánh chính xác thực tế, từ đó ngăn chặn hoàn toàn hành vi trục lợi của người dùng.

Phiên 1	Phiên 2	Giải thích
SQL> SELECT SoDu INTO v_SoDuHienTai FROM VI_CA_NHAN WHERE MaTaiKhoan = 1; DBMS_OUTPUT.PUT_LINE('Số dư hiện tại là: ' v_SoDuHienTai); 	Không có hoạt động	Phiên 1 truy vấn số dư trong ví của tài khoản 1

Số dư hiện tại là: 10000000		
SQL> UPDATE VI_CA_NHAN SET SoDu = v_SoDuHienTai - 10000000 WHERE MaTaiKhoan = 1;	Không có hoạt động	Phiên 1 bắt đầu một transaction bằng cách cập nhật số dư trong ví của tài khoản 1. Số dư mới bằng số dư hiện tại trừ đi 10.000.000đ. Ngay lúc này, dòng dữ liệu có mã tài khoản bằng 1 trong Ví Cá Nhân bị khoá bởi transaction 1. Mức cô lập mặc định cho transaction này là READ COMMITTED
Không có hoạt động	SQL> SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;	Phiên 2 bắt đầu một transaction 2 và thiết lập tường minh mức cô lập SERIALIZABLE
Không có hoạt động	SQL> SELECT SoDu INTO v_SoDuHienTai FROM VI_CA_NHAN WHERE MaTaiKhoan = 1; DBMS_OUTPUT.PUT_LI NE('Số dư hiện tại là: ' v_SoDuHienTai);	Transaction 2 truy vấn số dư ví của tài khoản 1. Oracle Database sử dụng tính nhất quán đọc để hiển thị, do thao tác cập nhật của transaction 1 chưa được commit nên số dư hiển thị ở phiên 2 là số dư trước khi transaction 1 bắt đầu.

	Số dư hiện tại là: 10000000	
Không có hoạt động	SQL> UPDATE VI_CA_NHAN SET SoDu = v_SoDuHienTai + 100000 WHERE MaTaiKhoan = 1; — Câu lệnh không trả về	Transaction 2 cố gắng cập nhật số dư của tài khoản 1 tăng thêm 100.000, tuy nhiên dòng dữ liệu này đã bị khoá bởi transaction 1, điều này tạo ra một xung đột ghi. Transaction 2 phải chờ đến khi transaction 1 kết thúc mới được thực thi tiếp
SQL> COMMIT;	Không có hoạt động	Transaction 1 commit và kết thúc
Không có hoạt động	ORA-08177: can't serialize access for this transaction	Khi Transaction 1 thực hiện lệnh commit, nó giải phóng khóa trên dòng dữ liệu và thiết lập một trạng thái mới cho số dư tài khoản. Lúc này Transaction 2 (đang ở trạng thái chờ) được tiếp tục thực thi lệnh cập nhật. Tuy nhiên, do Transaction 2 đang chạy ở mức cô lập SERIALIZABLE , Oracle phát hiện dòng dữ liệu mà Transaction 2 đang cố gắng cập nhật đã bị sửa đổi bởi

		một giao dịch khác và giao dịch đó commit sau thời điểm Transaction 2 bắt đầu. Để bảo vệ tính toàn vẹn của mức cô lập, Oracle từ chối thao tác ghi và trả về lỗi ORA-08177.
Không có hoạt động	SQL> ROLLBACK	Phiên 2 ROLLBACK transaction 2 và kết thúc transaction này.
Không có hoạt động	SQL> SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;	Phiên 2 bắt đầu một transaction 3 và thiết lập tường minh mức cô lập SERIALIZABLE
Không có hoạt động	SQL> SELECT SoDu INTO v_SoDuHienTai FROM VI_CA_NHAN WHERE MaTaiKhoan = 1; DBMS_OUTPUT.PUT_LINE('Số dư hiện tại là: ' v_SoDuHienTai); _____ Số dư hiện tại là: 0	Transaction 3 truy vấn số dư ví của tài khoản 1. Việc cập nhật số dư tài khoản 1 của transaction 1 đã được hiển thị
Không có hoạt động	SQL> UPDATE VI_CA_NHAN SET SoDu = v_SoDuHienTai +	Transaction 3 cập nhật số dư tài khoản 1 tăng thêm 100.000. Vì việc cập nhật

	100000 WHERE MaTaiKhoan = 1; 1 row affected.	bởi transaction 1 đã được commit trước khi transaction 3 bắt đầu nên vấn đề truy cập tuần tự được tránh
Không có hoạt động	SQL> COMMIT;	Phiên 2 commit việc cập nhật mà không gặp vấn đề nào, kết thúc transaction 3

2. Non-repeatable read

Tình huống xảy ra lỗi: Hệ thống bán vé sự kiện Ve'ryGood có hỗ trợ tính năng thị trường thứ cấp, cho phép người dùng tự do nhượng lại vé của mình cho người khác với mức giá tùy chỉnh. Một khách hàng (người mua) đang tìm kiếm vé và chọn được một tấm vé với giá niêm yết ban đầu là 500.000đ. Khách hàng này tiến hành nhấn chọn mua, hệ thống truy vấn giá vé và hiển thị tổng tiền cần thanh toán trên màn hình xác nhận.

Tuy nhiên, trong lúc khách hàng đang xem xét các bước thanh toán cuối cùng, người bán vé vào hệ thống để cập nhật giá bán lại của tấm vé đó từ 500.000đ lên một mức giá 4.900.000đ. Kết quả do lỗi truy xuất Non-repeatable Read ở phía hệ thống, khi khách hàng bấm xác nhận thanh toán truy vấn trừ tiền của hệ thống lại tiến hành đọc lại dữ liệu bảng giá vé thêm một lần nữa. Lúc này hệ thống đọc được mức giá mới nhất mà người bán vừa cập nhật, khiến khách hàng bị trừ mất 4.900.000đ trong khi ban đầu xem giá niêm yết là 500.000đ.

Giả sử tài khoản của người mua vé có mã tài khoản là 1 với số dư ví cá nhân ban đầu là 5.000.000đ, tấm vé được giao dịch có mã vé là 1. Phiên 1 là tiến trình xử lý quá trình hiển thị và thanh toán mua vé của khách hàng, Phiên 2 là tiến trình người bán cập nhật lại giá vé. Chi tiết quá trình xử lý bên dưới hệ thống dẫn đến lỗi được mô phỏng như sau:

Phiên 1	Phiên 2	Giải thích
SQL> SELECT SoDu FROM VI_CA_NHAN WHERE MaTaiKhoan = 1; SoDu _____ 5.000.000	Không có hành động	Phiên 1 truy vấn số dư trong ví cá nhân của tài khoản 1
SQL> SET TRANSACTION ISOLATION LEVEL READ COMMITTED;	Không có hành động	Phiên 1 bắt đầu transaction 1 và thiết lập tường minh mức cô lập READ COMMITTED
SQL> SELECT GiaBanLai FROM VE MaVe = 1; GiaBanLai _____ 500.000	Không có hành động	Phiên 1 truy vấn giá bán lại của vé có mã vé là 1
Không có hành động	SQL> UPDATE VE SET GiaBanLai = 4900000 WHERE MaVe = 1; 1 row affected	Phiên 2 bắt đầu transaction 2 bằng cách cập nhật giá bán lại của tấm vé số 1 từ 500.000đ thành 4.900.000đ. Mức cô lập mặc định cho transaction này là READ COMMITTED

Không có hành động	SQL> COMMIT;	Transaction 2 commit và kết thúc
UPDATE VI_CA_NHAN SET SoDu = SoDu - (SELECT GiaBanLai FROM VE WHERE MaVe = 1) WHERE MaTaiKhoan = 1;	Không có hành động	Transaction 1 thực hiện cập nhật cập nhật số dư trong ví cá nhân của tài khoản 1, số dư mới bằng số dư cũ trừ đi tiền vé. Mặc dù trước đó Phiên 1 thấy giá vé là 500.000đ, nhưng lệnh cập nhật này bắt đầu một chu kỳ đọc mới. Subquery sẽ truy vấn lại bảng VE. Vì đang ở mức cô lập READ COMMITTED , truy vấn con sẽ trả về số 4.900.000 do transaction 2 cập nhật.
SQL> COMMIT;	Không có hành động	Transaction 1 commit và kết thúc
SQL> SELECT SoDu FROM VI_CA_NHAN WHERE MaTaiKhoan = 1; SoDu _____ 100.000	Không có hành động	Phiên 1 truy vấn lại số dư trong tài khoản 1 thì phát hiện bất thường

Để khắc phục lỗi hổng Non-repeatable Read trong kịch bản thanh toán này, giải pháp được đưa ra là thiết lập tường minh mức cô lập của giao dịch mua vé (Phiên 1) lên **SERIALIZABLE**. Thông qua việc duy trì một phiên bản dữ liệu cố định không bị tác động bởi các giao dịch commit song song, Oracle Database đảm bảo khách hàng chỉ phải thanh toán đúng với số tiền đã được hiển thị trên màn hình ban đầu, loại bỏ hoàn toàn rủi ro thất thoát tài chính do hiện tượng Non – repeatable Read gây ra.

Phiên 1	Phiên 2	Giải thích
SQL> SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;	Không có hành động	Phiên 1 bắt đầu transaction 1 và thiết lập tường minh mức cô lập SERIALIZABLE
SQL> SELECT SoDu FROM VI_CA_NHAN WHERE MaTaiKhoan = 1; SoDu _____ 5.000.000	Không có hành động	Phiên 1 truy vấn số dư trong ví cá nhân của tài khoản 1
SQL> SELECT GiaBanLai FROM VE MaVe = 1; GiaBanLai _____ 500.000	Không có hành động	Phiên 1 truy vấn giá bán lại của vé có mã vé là 1
Không có hành động	SQL> UPDATE VE SET GiaBanLai = 4900000	Phiên 2 bắt đầu transaction 2 bằng cách cập nhật giá bán lại của tám vé số 1 từ

	WHERE MaVe = 1; 1 row affected	500.000đ thành 4.900.000đ. Mức cô lập mặc định cho transaction này là READ COMMITTED
Không có hành động	SQL> COMMIT;	Transaction 2 commit và kết thúc
UPDATE VI_CA_NHAN SET SoDu = SoDu - (SELECT GiaBanLai FROM VE WHERE MaVe = 1) WHERE MaTaiKhoan = 1;	Không có hành động	Transaction 1 thực hiện cập nhật cập nhật số dư trong ví cá nhân của tài khoản 1, số dư mới bằng số dư cũ trừ đi tiền vé. Vì đang ở mức cô lập SERIALIZABLE , truy vấn con sẽ trả về 500.000đ như lúc bắt đầu transaction 1 mà không trả về 4.900.000đ dù cho transaction 2 đã commit.
SQL> COMMIT;	Không có hành động	Transaction 1 commit và kết thúc
SQL> SELECT SoDu FROM VI_CA_NHAN WHERE MaTaiKhoan = 1; SoDu _____	Không có hành động	Phiên 1 truy vấn lại số dư trong tài khoản 1.
		4.500.000

3. Phantom read

Tình huống xảy ra lỗi: Hệ thống bán vé Ve'ryGood có quy trình quyết toán doanh thu tự động vào cuối ngày để tổng hợp dòng tiền cho các Nhà tổ chức sự kiện. Quy trình này bao gồm hai bước quan trọng: đầu tiên là truy vấn danh sách chi tiết các giao dịch thành công để xuất file đối soát (Excel), và sau đó là tính tổng doanh thu để ghi nhận vào bảng lịch sử quyết toán của cơ sở dữ liệu.

Lỗi Phantom Read xảy ra khi hệ thống đang ở giữa hai bước này thì có một giao dịch thanh toán trễ từ cổng thanh toán gửi tín hiệu thành công về hệ thống. Do mức cô lập mặc định là Read Committed không khóa vùng dữ liệu theo điều kiện truy vấn, lệnh tổng hợp dữ liệu ở bước sau sẽ quét toàn bộ bảng và nhận diện thêm "giao dịch bóng ma" vừa mới được commit. Điều này dẫn đến một sai sót nghiêm trọng về mặt kế toán: số tiền tổng hợp trong cơ sở dữ liệu lớn hơn tổng số tiền của các giao dịch chi tiết trong file đối soát, gây khó khăn cho việc minh bạch tài chính với Nhà tổ chức.

Giả sử trong ngày hiện tại đã có 2 giao dịch thành công với tổng số tiền là 500.000đ. Phiên 1 là tiến trình quyết toán tự động của hệ thống, Phiên 2 đại diện cho tín hiệu Webhook từ cổng thanh toán báo về cho một đơn hàng mới trị giá 1.000.000đ. Chi tiết quá trình thực thi dẫn đến sự không nhất quán này như sau:

Phiên 1	Phiên 2	Giải thích
SQL> SET TRANSACTION ISOLATION LEVEL READ COMMITTED;	Không có hoạt động	Phiên 1 bắt đầu transaction 1 và thiết lập tường minh mức cô lập READ COMMITED
SELECT gd.MaGiaoDich, gd.SoTien FROM GIAO_DICH gd JOIN DON_MUA dm ON	Không có hoạt động	Phiên 1 truy vấn mã giao dịch và số tiền

<pre>gd.MaDonMua = dm.MaDonMua JOIN SU_KIEN sk ON dm.MaSuKien = sk.MaSuKien WHERE (sk.MaNhaToChuc = 1001) AND (gd.TrangThai = N'Thành công') AND (gd.ThoiGianTao >= TRUNC(CURRENT_TIMESTAMP)) AND (gd.ThoiGianTao < TRUNC(CURRENT_TIMESTAMP) + 1);</pre> <table><tr><th>MaGiaoDich</th><th>SoTien</th></tr><tr><td>1001</td><td>200000</td></tr><tr><td>1002</td><td>300000</td></tr></table>	MaGiaoDich	SoTien	1001	200000	1002	300000		của tất cả các vé đã được thanh toán thành công trong ngày hôm nay, thuộc về các sự kiện do nhà tổ chức có mã 1001 quản lý.
MaGiaoDich	SoTien							
1001	200000							
1002	300000							
Không có hoạt động	<pre>SQL> INSERT INTO GIAO_DICH (MaGiaoDich, MaDonMua, SoTien, TrangThai, ThoiGianTao) VALUES (1003, 1003, 1000000, N'Thành công', CURRENT_TIMESTAMP);</pre>	Cổng thanh toán gửi Webhook báo đơn hàng số 3 đã thanh toán thành công. Phiên 2 bắt đầu transaction 2 bằng việc thêm một giao dịch mới.						

	SQL> COMMIT;	Transaction 2 commit và kết thúc.
SQL> INSERT INTO LICH_SU QUYET_TOAN (MaNhaToChuc, KyQT, TongDoanhThu, PhiNenTang, SoTienChuyen) SELECT 1001, 'Kỳ hiện tại', SUM(gd.SoTien), SUM(gd.SoTien) * 0.05, SUM(gd.SoTien) * 0.95 FROM GIAO_DICH gd JOIN DON_MUA dm ON gd.MaDonMua = dm.MaDonMua JOIN SU_KIEN sk ON dm.MaSuKien = sk.MaSuKien WHERE (sk.MaNhaToChuc = 1001) AND (gd.TrangThai = N'Thành công') AND (gd.ThoiGianTao >= TRUNC(CURRENT_TIMESTAMP)) AND (gd.ThoiGianTao < TRUNC(CURRENT_TIMESTAMP) + 1);	Không có hoạt động	Transaction 1 thực hiện quyết toán cho nhà tổ chức 1001 (lấy mã nhà tổ chức, kỳ quyết toán, tổng doanh thu, phí nền tảng và số tiền chuyển của kỳ quyết toán ngày hôm nay thêm vào lịch sử quyết toán)
SQL> COMMIT;	Không có hoạt động	Transaction 1 commit và kết thúc

SQL> SELECT TongDoanhThu FROM LICH_SU QUYET_TOAN WHERE MaNhaToChuc = 1001 AND KyQT = N'Kỳ hiện tại'; TongDoanhThu 1.500.000	Không có hoạt động	Phiên 1 thực hiện truy vấn lại tổng doanh thu của lần quyết toán vừa xong. File đối soát xuất ở bước đầu chỉ ghi nhận 500.000đ (2 giao dịch), nhưng bản ghi quyết toán cuối cùng lại là 1.500.000đ (3 giao dịch). Tổng doanh thu đã bao gồm luôn cả giao dịch bóng ma (giao dịch 1003) vừa xuất hiện.
--	--------------------	---

Để khắc phục triệt để lỗi hồng Phantom Read gây sai lệch số liệu trong quá trình quyết toán, giải pháp là thiết lập mức cô lập của tiến trình quyết toán hệ thống (Phiên 1) lên mức **SERIALIZABLE**. Thông qua việc sử dụng mức cô lập Serializable, ứng dụng duy trì được tính bất biến của tập kết quả truy vấn, đảm bảo độ tin cậy tuyệt đối cho các tiến trình kế toán và báo cáo tổng hợp dữ liệu quy mô lớn.

Phiên 1	Phiên 2	Giải thích
SQL> SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;	Không có hoạt động	Phiên 1 bắt đầu transaction 1 và thiết lập mức cô lập tường minh SERIALIZABLE . Kể từ lúc này, Oracle tạo ra một "ảnh chụp" dữ liệu tại thời điểm hiện tại cho Phiên 1.
SELECT gd.MaGiaoDich, gd.SoTien FROM GIAO_DICH gd JOIN DON_MUA dm ON gd.MaDonMua = dm.MaDonMua JOIN SU_KIEN sk ON dm.MaSuKien = sk.MaSuKien WHERE (sk.MaNhaToChuc = 1001) AND (gd.TrangThai = N'Thành công') AND (gd.ThoiGianTao >= TRUNC(CURRENT_TIMESTAMP)) AND (gd.ThoiGianTao < TRUNC(CURRENT_TIMESTAMP) + 1); MaGiaoDich SoTien _____	Không có hoạt động	Phiên 1 truy vấn mã giao dịch và số tiền của tất cả các vé đã được thanh toán thành công trong ngày hôm nay, thuộc về các sự kiện do nhà tổ chức có mã 1001 quản lý.

1001	200000		
1002	300000		
Không có hoạt động	SQL> INSERT INTO GIAO_DICH (MaGiaoDich, MaDonMua, SoTien, TrangThai, ThoiGianTao) VALUES (1003, 1003, 1000000, N'Thành công', CURRENT_TIMESTAMP);	Cổng thanh toán gửi Webhook báo đơn hàng số 3 đã thanh toán thành công. Phiên 2 bắt đầu transaction 2 bằng việc thêm một giao dịch mới.	
Không có hoạt động	SQL> COMMIT;	Transaction 2 commit và kết thúc.	
SQL> INSERT INTO LICH_SU QUYET_TOAN (MaNhaToChuc, KyQT, TongDoanhThu, PhiNenTang, SoTienChuyen) SELECT 1001, 'Kỳ hiện tại', SUM(gd.SoTien), SUM(gd.SoTien) * 0.05, SUM(gd.SoTien) * 0.95 FROM GIAO_DICH gd JOIN DON_MUA dm ON gd.MaDonMua = dm.MaDonMua JOIN SU_KIEN sk ON dm.MaSuKien = sk.MaSuKien WHERE (sk.MaNhaToChuc = 1001) AND (gd.TrangThai = N'Thành	Không có hoạt động	Transaction 1 thực hiện quyết toán cho nhà tổ chức 1001 (lấy mã nhà tổ chức, kỳ quyết toán, tổng doanh thu, phí nền tảng và số tiền chuyển của kỳ quyết toán ngày hôm nay thêm vào lịch sử quyết toán). Mặc dù Phiên 2 đã commit, nhưng do đang ở mức	

công') AND (gd.ThoiGianTao >= TRUNC(CURRENT_TIMESTAMP)) AND (gd.ThoiGianTao < TRUNC(CURRENT_TIMESTAMP) + 1);		SERIALIZABLE , Oracle sử dụng cơ chế MVCC để lọc bỏ các dòng dữ liệu tương lai, chỉ lấy dữ liệu hợp lệ tại thời điểm Phiên 1 bắt đầu.
SQL> COMMIT;	Không có hoạt động	Transaction 1 commit và kết thúc.
SQL> SELECT TongDoanhThu FROM LICH_SU QUYET_TOAN WHERE MaNhaToChuc = 1001 AND KyQT = N'Kỳ hiện tại'; TongDoanhThu _____ 500.000	Không có hoạt động	Phiên 1 thực hiện truy vấn lại tổng doanh thu của lần quyết toán vừa xong. Vấn đề "bóng ma" đã được giải quyết. Số tiền quyết toán trùng khớp hoàn toàn với danh sách chi tiết ở bước đầu (500.000đ). Giao dịch mới của Phiên 2 sẽ được xử lý vào kỳ quyết toán tiếp theo.

4. Deadlock

Tình huống xảy ra lỗi: Hệ thống Ve'ryGood cho phép người dùng thực hiện giao dịch mua bán vé trên thị trường thứ cấp. Một tình huống bế tắc (Deadlock) phát sinh khi hai người dùng thực hiện mua vé chéo của nhau tại cùng một thời điểm. Cụ thể, tài khoản 1001 đang tiến hành mua một vé bán lại trị giá 500.000đ từ tài khoản 1002. Cùng lúc đó, tài khoản 1002 cũng đang thực hiện mua một vé khác trị giá 300.000đ từ tài khoản 1001.

Để đảm bảo tính nguyên tử của giao dịch, mỗi phiên làm việc phải thực hiện đồng thời hai thao tác: trừ tiền người mua và cộng tiền cho người bán trên bảng ví cá nhân. Lỗi Deadlock xảy ra khi mỗi phiên đã kịp thời chiếm giữ khóa độc quyền trên dòng dữ liệu của chính mình (người mua) nhưng lại bị rơi vào trạng thái chờ khi cố gắng truy cập vào dòng dữ liệu của đối phương (người bán) vốn đã bị phiên kia khóa trước đó. Sự cạnh tranh tài nguyên này tạo thành một vòng lặp chờ đợi khép kín, khiến cả hai giao dịch bị đình trệ vô thời hạn nếu không có sự can thiệp từ hệ quản trị cơ sở dữ liệu.

Giả sử tài khoản 1001 có số dư ban đầu là 600.000đ và tài khoản 1002 có số dư là 900.000đ. Phiên 1 xử lý quá trình mua vé của tài khoản 1001, Phiên 2 xử lý quá trình mua vé của tài khoản 1002. Chi tiết quá trình thực thi dẫn đến bế tắc bên dưới hệ thống như sau:

Phiên 1	Phiên 2	Giải thích																
SQL> SELECT MaTaiKhoan, SoDu FROM VI_CA_NHAN WHERE MaTaiKhoan IN (1001, 1002); <table><tr><td>MaTaiKhoan</td><td>SoDu</td></tr><tr><td>-----</td><td>-----</td></tr><tr><td>1001</td><td>600000</td></tr><tr><td>1002</td><td>900000</td></tr></table>	MaTaiKhoan	SoDu	-----	-----	1001	600000	1002	900000	SQL> SELECT MaTaiKhoan, SoDu FROM VI_CA_NHAN WHERE MaTaiKhoan IN (1001, 1002); <table><tr><td>MaTaiKhoan</td><td>SoDu</td></tr><tr><td>-----</td><td>-----</td></tr><tr><td>1001</td><td>600000</td></tr><tr><td>1002</td><td>900000</td></tr></table>	MaTaiKhoan	SoDu	-----	-----	1001	600000	1002	900000	Phiên 1 và phiên 2 cùng truy vấn số dư trong ví của tài khoản 1001 và 1002
MaTaiKhoan	SoDu																	
-----	-----																	
1001	600000																	
1002	900000																	
MaTaiKhoan	SoDu																	
-----	-----																	
1001	600000																	
1002	900000																	
SQL> UPDATE VI_CA_NHAN SET SoDu = SoDu - 500000 WHERE MaTaiKhoan = 1001; 1 row affected	SQL> UPDATE VI_CA_NHAN SET SoDu = SoDu - 300000 WHERE MaTaiKhoan = 1002; 1 row affected	Phiên 1 bắt đầu transaction 1 và cập nhật số dư trong ví của tài khoản 1001 trừ đi 500.000đ. Phiên 2 bắt đầu transaction 2 và cập nhật số dư trong ví của của tài khoản 1002 trừ đi 300.000đ. Không có vấn đề nào phát sinh vì mỗi transaction chỉ khoá đúng dòng dữ liệu mà nó cập nhật																
SQL> UPDATE VI_CA_NHAN SET SoDu = SoDu + 500000 WHERE	SQL> UPDATE VI_CA_NHAN SET SoDu = SoDu + 300000 WHERE MaTaiKhoan = 1001;	Transaction 1 cố cập nhật số dư tăng thêm 500.000đ trong ví của tài khoản 1002, tuy nhiên dòng dữ																

MaTaiKhoan = 1002; — prompt does not return	— prompt does not return	liệu này đã bị khoá bởi transaction 2. Transaction 1 cố cập nhật số dư tăng thêm 300.000đ trong ví của tài khoản 1001, tuy nhiên dòng dữ liệu này đã bị khoá bởi transaction 2. Deadlock xảy ra vì không có transaction nào lấy được tài nguyên để tiếp tục thực thi và kết thúc. Cho dù mỗi transaction chờ bao lâu thì các khoá xung đột vẫn bị transaction còn lại giữ.
ORA-00060: deadlock detected while waiting for resource SQL>	Không có hoạt động	Transaction 1 báo hiệu tình trạng deadlock và rollback câu lệnh cập nhật số dư trong ví của tài khoản 1002, tuy nhiên lệnh cập nhật trừ đi số dư trong ví của tài khoản 1001 thì không được rollback.

SQL> SELECT MaTaiKhoan, SoDu FROM VI_CA_NHAN WHERE MaTaiKhoan IN (1001, 1002); MaTaiKhoan SoDu ----- 1001 100000 1002 900000	Không có hoạt động	Phiên 1 truy vấn số dư trong ví của tài khoản 1001 và 1002
SQL> ROLLBACK	Không có hoạt động	Transaction 1 rollback về trạng thái trước khi cập nhật trừ số dư trong tài khoản 1001
Không có hoạt động	1 row affected SQL>	Do transaction 1 kết thúc, khoá trên dòng dữ liệu số dư ví tài khoản 1001 được giải phóng, transaction 2 tiếp tục cập nhật số dư thành công.
Không có hoạt động	SQL> COMMIT;	Transaction 2 commit và kết thúc
Không có hoạt động	SQL> SELECT MaTaiKhoan, SoDu FROM VI_CA_NHAN WHERE MaTaiKhoan IN (1001, 1002);	Phiên 2 truy vấn số dư trong ví của tài khoản 1001 và 1002

	MaTaiKhoan SoDu ----- - ----- 1001 900000 1002 600000	
SQL> UPDATE VI_CA_NHAN SET SoDu = SoDu - 500000 WHERE MaTaiKhoan = 1001; 1 row affected	Không có hoạt động	Phiên 1 bắt đầu transaction 3 và cập nhật số dư trong ví của tài khoản 1001 trừ đi 500.000đ.
SQL> UPDATE VI_CA_NHAN SET SoDu = SoDu + 500000 WHERE MaTaiKhoan = 1002; 1 row affected	Không có hoạt động	Transaction 3 cập nhật số dư tăng thêm 500.000đ trong ví của tài khoản 1002.
SQL> COMMIT;	Không có hoạt động	Transaction 3 commit và kết thúc
SQL> SELECT MaTaiKhoan, SoDu FROM VI_CA_NHAN WHERE MaTaiKhoan IN (1001, 1002); MaTaiKhoan SoDu ----- - -----	Không có hoạt động	Phiên 2 truy vấn số dư trong ví của tài khoản 1001 và 1002

1001	400000		
1002	1100000		

IV. Tổng kết

Quản lý truy xuất đồng thời luôn là bài toán nhằm duy trì tính nhất quán và toàn vẹn dữ liệu trong các hệ thống cơ sở dữ liệu đa người dùng. Qua seminar này, nhóm đã làm rõ cách Oracle Database tiếp cận và giải quyết vấn đề bằng kiến trúc Kiểm soát truy xuất đồng thời đa phiên bản (MVCC). Việc kết hợp khéo léo giữa số thứ tự thay đổi hệ thống và dữ liệu hoàn tác giúp Oracle cung cấp tính nhất quán đọc mạnh mẽ, hạn chế việc sử dụng khóa tràn lan để đảm bảo hiệu năng tổng thể của hệ thống. Qua phân tích các kịch bản thực tế, có thể thấy mức cô lập mặc định Read Committed mang lại độ khả dụng cao, nhưng ứng dụng phải đối mặt với các lỗi hổng nghiêm trọng như Lost Update, Non-repeatable Read hay Phantom Read. Giải pháp thiết lập mức cô lập Serializable đã chứng minh được hiệu quả tuyệt đối trong việc ngăn chặn các hiện tượng trên nhờ duy trì một ảnh chụp dữ liệu bất biến xuyên suốt vòng đời giao dịch. Tuy nhiên, sự an toàn này đòi hỏi sự đánh đổi: ứng dụng phải được thiết kế để bắt và xử lý các ngoại lệ chặn ghi từ Oracle (lỗi ORA-08177), chủ động rollback và thực thi lại giao dịch khi cần thiết.

Bên cạnh đó, hiện tượng Deadlock là một rủi ro tất yếu trong các kịch bản cập nhật chéo phức tạp. Cơ chế tự động phát hiện vòng lặp bế tắc và phá vỡ bằng lỗi ORA-00060 của Oracle giúp hệ thống không bị treo vô thời hạn, nhưng đồng thời cũng yêu cầu các nhà phát triển phải có kịch bản xử lý rollback an toàn ở tầng ứng dụng.

Tóm lại, không có một mức cô lập nào là hoàn hảo cho mọi trường hợp. Để thiết kế một hệ thống tối ưu, chúng ta cần nắm vững nguyên lý hoạt động của các mức cô lập, từ đó đánh giá chính xác logic nghiệp vụ của từng Module (như thanh toán, hoàn tiền, quyết toán) để đưa ra chiến lược quản lý xung đột phù hợp cân bằng tính chính xác tuyệt đối của dữ liệu và hiệu năng vận hành thực tế.